Stream codes

Last modified: 2026-11-08 23:05:30 (PST)

A Huffman code gives each value a whole number of bits, so its expected length can exceed the entropy by nearly one bit per value. This page shows how to code a whole message at once, so that the waste is shared among all of its values.

Arithmetic coding

Definition 0.1 (Arithmetic code)**.** Let X be a discrete random variable whose values come in a fixed order, with probability mass function $\mathbb{p}$, and let $F(x) = s_{x'<x}\mathbb{p}(x')$ for the values x' before x. For a message $x = (x_1, \dots, x_n)$ of n independent copies of X, the **arithmetic code** is built as follows:

1. Start with the slice $[0, 1)$.
2. For $i = 1, \dots, n$: if the current slice is $[a, b)$, replace it with the part of $[a, b)$ that x_i owns, which is $[a + (b-a)F(x_i),\ a + (b-a)(F(x_i) + \mathbb{p}(x_i)))$.
3. Let $[a, a+w)$ be the final slice, and let $m = a + w/2$ be its midpoint. The codeword $c(x)$ is the first $\ell(x)$ bits of m after the point, where the length $\ell(x) = \lceil -\log_2 w \rceil + 1$ is part of this definition.

Example 0.1 (An arithmetic code for a short message)**.**
Take a source with the values a, b and c, in that order, and the probabilities $(0.5, 0.3, 0.2)$. The code below follows Definition 0.1 for the message a, c, b, a. It prints the slice after each round, then the codeword and its codeword length.

```
p <- c(a = 0.5, b = 0.3, c = 0.2)
cdf_before <- cumsum(p) - p
message_values <- c("a", "c", "b", "a")
n <- length(message_values)
lower <- 0
width <- 1
lowers <- numeric(n)
widths <- numeric(n)
for (i in seq_len(n)) {
  symbol <- message_values[i]
  lower <- lower + width * cdf_before[[symbol]]
  width <- width * p[[symbol]]
  lowers[i] <- lower
  widths[i] <- width
}
uppers <- lowers + widths
data.frame(
  symbol = message_values,
  lower = lowers,
  upper = uppers,
  width = widths
)
#>   symbol lower upper width
#> 1      a  0.00 0.500 0.500
#> 2      c  0.40 0.500 0.100
#> 3      b  0.45 0.480 0.030
#> 4      a  0.45 0.465 0.015
midpoint <- lower + width / 2
surprisal <- -log2(width)
n_digits <- ceiling(surprisal) + 1
positions <- seq_len(n_digits)
digits <- floor(midpoint * 2^positions) %% 2
codeword <- paste0(digits, collapse = "")
c(midpoint = midpoint, codeword_length = n_digits, surprisal = surprisal)
#>        midpoint codeword_length       surprisal
#>        0.457500        8.000000        6.058894
codeword
#> [1] "01110101"
```

The final slice has width 0.015, which is the product $0.5 \times 0.2 \times 0.3 \times 0.5$ of the probabilities of the four values. The codeword is 01110101, with 8 digits, which is the message's

surprisal of 6.059 bits rounded up to a whole number, plus one.

Lemma 0.1 (The intervals of an arithmetic code)**.** *In the arithmetic code for n independent copies of X, the final slice of a message $x = (x_1, \dots, x_n)$ has width $\mathbb{p}(x_1) \cdots \mathbb{p}(x_n)$, and the final slices of two different messages do not overlap.*

Proof

Proof. Each round replaces a slice of width $b - a$ by a slice of width $(b - a)\mathbb{p}(x_i)$. After n rounds the width is $\mathbb{p}(x_1) \cdots \mathbb{p}(x_n)$.
The parts that the values of X own are the slices $[F(x), F(x) + \mathbb{p}(x))$, which do not overlap, because each starts where the one before it ends. So the parts of one slice do not overlap either.
Let x and x' be two different messages, and let j be the first position where they differ. Before round j, both messages have the same slice. At round j they take different parts of it, which do not overlap. Later rounds keep each slice inside its own part, so the final slices do not overlap. □

Theorem 0.1 (Arithmetic codes are prefix codes)**.** *Let X be a discrete random variable whose values each have probability. Then the codewords of the arithmetic code for n independent copies of X form a prefix code on the messages of length n.*

Proof

Proof. Let x be a message, let $[a, a + w)$ be its final slice, and let $m = a + w/2$. Write $k = \ell(x)$, and let t be the number whose bits after the point are $c(x)$. Truncating m to k digits gives

$$t \le m < t + 2^{-k}.$$

Because $k \ge -\log_2 w + 1$, we have $2^{-k} \le w/2$. So $t > m - 2^{-k} \ge m - w/2 = a$, and $t + 2^{-k} \le m + w/2 = a + w$. This shows that the slice $[t, t + 2^{-k})$, which holds the numbers whose bits begin with $c(x)$, lies inside the final slice of x.
Suppose that $c(x)$ were the beginning of $c(x')$ for two different messages x and x'. Every number whose digits begin with $c(x')$ also begins with $c(x)$, so the slice of $c(x')$ lies inside the slice of $c(x)$. The first lies inside the final slice of x', and the second inside the final slice of x, so these two final slices would share a point. Lemma 0.1 rules that out, so no codeword is the beginning of another. □

Theorem 0.2 (The expected length of an arithmetic code)**.** *Let X be a discrete random variable that takes a finite number of values, each with probability, and let $\mathbb{H}(X)$ be its entropy with base-2 logarithms. Then the arithmetic code for n independent copies of X has expected length*

$$L < n\mathbb{H}(X) + 2,$$

which is fewer than $\mathbb{H}(X) + 2/n$ bits per copy.

Proof

Proof. By Lemma 0.1, the final slice of a message $x = (x_1, \dots, x_n)$ has width $w = \mathbb{p}(x_1) \cdots \mathbb{p}(x_n)$. So the codeword length satisfies

$$\ell(x) = \lceil -\log_2 w \rceil + 1 < -\log_2 w + 2 = \sum_{i=1}^{n} \bigl(-\log_2 \mathbb{p}(x_i)\bigr) + 2.$$

Taking the average over messages, and using that each copy X_i has the expectation $\mathbb{E}\bigl[-\log_2 \mathbb{p}(X_i)\bigr] = \mathbb{H}(X)$, gives $L < n\mathbb{H}(X) + 2$. Dividing by n gives the length per copy. □

Remark 0.1 (Where arithmetic codes come from). Cover and Thomas (2006, sec. 5.9) give Shannon–Fano–Elias coding, which assigns each value a codeword from the midpoint of its slice of $[0, 1)$, in the way that Definition 0.1 does for a whole message. Cover and Thomas describe arithmetic coding, which finds the slice of a long message one value at a time, in section 13.3.

References

Cover, Thomas M., and Joy A. Thomas. 2006. *Elements of Information Theory.* 2nd ed. Wiley-Interscience. https://doi.org/10.1002/047174882X.